The handleSubmit even handler itself first uses the preventDefault call to stop the browser from triggering the default form submit action, which would refresh the page.

When you add a ref="name" property to an element in the render method, it allows you to quickly get a reference to that property using this.refs. Since we gave our input field a ref of "query" we can now access it as this.refs.query.

Here we are obtaining the value of the input field, and passing it to the queryChange function that seems to be a property as it is available in the this.props object.

What this means is that when we create an instance of this SearchBox component, we will need to provide a function via the queryChange property. This function will then by called when the query changes, and will be provided the query string.

Instead of listening for form submits, you can listen for text change events get live search results.

Connecting Everything

Now we need to create one final component that will connect everything together. Since we've already done a lot of the heavy lifting inside the application, the main app is quite small:

```
var App = React.createClass({
 getInitialState: function() {
  return {data: []};
 },
 handleQueryChange: function(query){
  var url = "https://omdbapi.com/?type=movie&s=" + query;
  $.get(url, function(data) {
   this.setState({data: data['Search']});
  }.bind(this));
 },
 render: function() {
  return (<div>
        <SearchBox queryChange={this.handleQueryChange} />
        <MovieList movies={this.state.data} />
       </div>);
 }
});
```

The render function is quite simple thanks to our existing components. It simply includes the SearchBox and MovieList components. The queryChange property of the SearchBox is bound to the handleQueryChange method of our component and the movies property of our MovieList is bound to the data stored in our state.

When someone submits a query using our SearchBox component the handleQueryChange method is called. This method is provided the submitted query, and this method in turn uses the OMDB API get the movies matching the query.

Here we are using jQuery's convenient Ajax APIs to perform the query. The OMDB API returns a JSON document with a single Array called "Search". Here we extract that array and use setState to update the data to be displayed by our component. Using setState to change the data automatically refreshes our component with new data.

At this point you have a fully working React-based application for searching for movies using the OMDB API and displaying the result.

Next Steps

Hopefully you've seen how easy it is to combine different components in React to work together and create complex UIs, and how easy it is to reuse the components you've made.

React's component model is generic enough that it is also usable for building native applications with React Native and graphics-driven apps with react-canvas.

Of course with the Virtual DOM being a core part of React, you can also expect better performance that most other similar UI libraries and frameworks. In fact since the DOM is virtual, it's even possible to render React components to HTML on the server side.

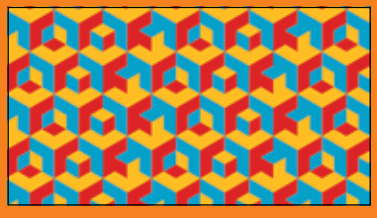
To find out more about React, you can visit: https://facebook.github.io/react/ You can also learn about the OMDB APIs we have used in our project at: http://omdbapi.com/>

*bread crumbs

*React Router

>>Now that you have your components set up using React, are you looking for a router to handle navigation within your app? What you need in react-router.

http://dgit.in/ReactRouter



*React Art

>>React is powerful enough to work with other models that the DOM, and this is a good example of how react can be used with the HTML canvas.

http://dgit.in/ReactArt

*Flux

>>Flux is another Facebook creation that aims to be a modern application architecture for apps built using React.

http://dgit.in/FbFlux